DEVOPS (24DC2006) · SKILL-BASED CASE STUDY

SeatSurge

Engineering a flash-sale concert-ticketing platform, and the DevOps pipeline that keeps it standing when every fan clicks “buy” at once

Project report

Submitted by: Isaac Benjamin David
Enrollment No.: _____
Programme: B.Tech, Semester VII
Course: DevOps (24DC2006)
Repository: github.com/Turbo-Soggy/seatsurge
Date: November 2026

Stack: GitHub Actions · Docker · Kubernetes (kind) · Helm · Terraform · Azure AKS · Prometheus · Grafana · Loki · Trivy · CodeQL · Sealed Secrets

Contents

1. Abstract 3
2. Problem statement and objectives 4
2.1 The problem 4
2.2 Objectives 4
2.3 Scope 4
3. System architecture 5
3.1 How a seat is sold exactly once 5
4. Version control and collaboration 7
4.1 Branching strategy: GitFlow-lite 7
4.2 Pull-request workflow and team practices 7
4.3 Merge-conflict case study 8
5. CI/CD pipeline automation 10
5.1 Continuous integration (every PR and push) 10
5.2 Continuous delivery (main only) 10
6. Containerization and orchestration 13
6.1 Optimized multi-stage Dockerfiles 13
6.2 Kubernetes objects (Helm chart) 13
6.3 Scaling and zero-downtime rolling update under load 13
6.4 Rollback 14
7. Infrastructure as code, cloud, monitoring and security 16
7.1 Terraform: the whole platform from one command 16
7.2 Monitoring, logging and alerting 16
7.3 Secrets management 17
7.4 Security scanning (DevSecOps) 17
8. Results 19
9. Defects caught by the pipeline, and how they were fixed 20
10. Rubric alignment 21
Pipeline execution: live demonstration script 21
11. Conclusion and future work 22
Future work 22
References 22

1. Abstract

SeatSurge is a small but real microservice system for a concert “ticket drop”: thousands of fans race for the same seats in the first minutes of a sale. The scenario is deliberately hostile to naive engineering. Traffic jumps from near zero to a stampede, two fans must never buy the same seat, and the team must be able to ship fixes in the middle of the sale without taking the site down.

Around three services (React web front-end, Python FastAPI inventory, Node.js checkout) the project implements a complete DevOps lifecycle: GitFlow-lite collaboration with protected branches and pull requests; a GitHub Actions CI pipeline with linting, static analysis, 49 unit tests, multi-stage container builds, a blocking vulnerability gate, SBOMs and registry publishing; a multi-stage CD pipeline (ephemeral dev cluster → staging → manual approval → production) with automatic rollback; Kubernetes deployment through a Helm chart with autoscaling and zero-downtime rolling updates; the entire platform provisioned with Terraform (plus an Azure AKS module); and monitoring, logging and alerting with Prometheus, Grafana, Loki and Alertmanager.

The headline result: **release v1.1.0 was rolled out to production while 150 virtual fans were buying tickets. 14,330 requests, 0 failed, p95 latency 23 ms, the event sold out (1,004 seats, ₹41,18,996), and the inventory service autoscaled from 2 to 10 pods.** Along the way the pipeline itself caught 13 real defects (in code, workflows and infrastructure), each fixed through the normal pull-request process and documented in Section 9.

2. Problem statement and objectives

2.1 The problem

When a popular artist's tickets go on sale, an online box office faces four engineering problems at once:

- **Correctness under contention.** Many fans click the same front-row seat within milliseconds. Selling it twice is unacceptable.
- **Elasticity.** Load rises by orders of magnitude in seconds and falls just as fast. Fixed capacity is either too expensive or too small.
- **Safe change.** Bugs surface during the sale. The team must release fixes without downtime and roll back instantly if a release misbehaves.
- **Visibility.** Operators need to see traffic, latency, sales, errors and alerts in real time, not after the event.

2.2 Objectives

1. Build a polyglot microservice application that models the ticket drop realistically.
2. Automate build, test, static analysis, security scanning and artifact publishing for every change (CI).
3. Automate delivery through dev, staging and production with a human approval gate and automatic rollback (CD).
4. Package the services in optimized, non-root containers and run them on Kubernetes with autoscaling and rolling updates.
5. Provision all infrastructure as code (Terraform), including a cloud (Azure AKS) target.
6. Monitor the system with metrics, logs, dashboards and alerts, and manage secrets without ever committing plaintext.
7. Prove all of the above with real runs: pull requests, pipeline executions, load tests and screenshots.

2.3 Scope

Production in this project is a local Kubernetes cluster (kind, 3 nodes) built by Terraform on the developer machine, reached through a self-hosted GitHub Actions runner. The Azure AKS module is complete and validated in CI but not applied, because it needs a paid subscription. Payments use a mock gateway that declines about 2 % of cards to exercise the failure path.

3. System architecture

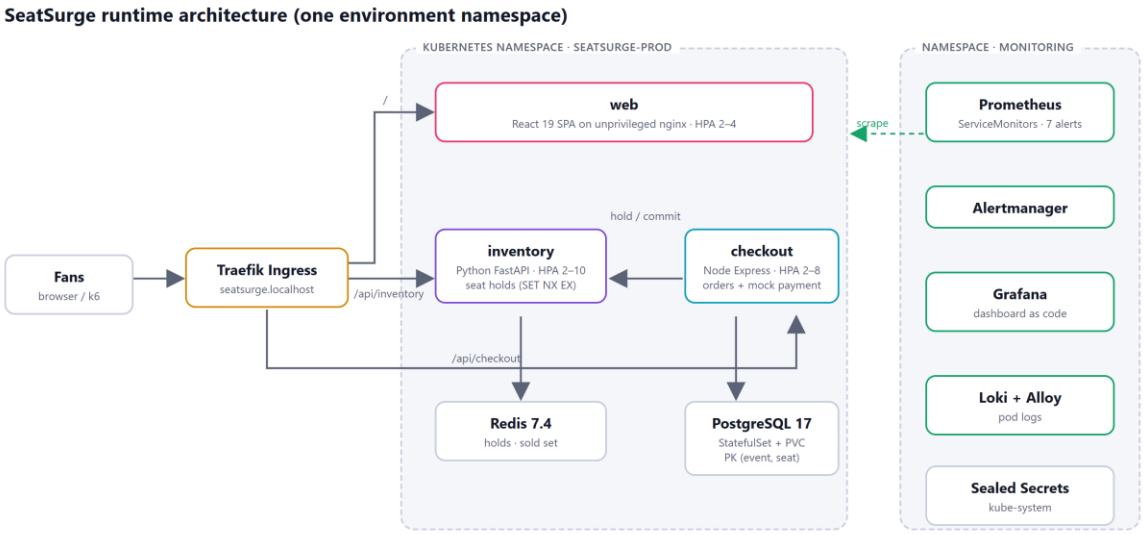


Figure 1. *Runtime architecture of one environment (production shown). Staging uses the same chart in its own namespace.*

Table 1. *Services and their responsibilities*

Service	Technology	Responsibility	Scales
web	React 19 + Vite + TypeScript, unprivileged nginx	Seat map, hold countdown, checkout form, ticket stub, live version chips	HPA 2–4
inventory	Python 3.13, FastAPI, Redis	Venue layout, availability, atomic seat holds	HPA 2–10
checkout	Node 24, Express 5, TypeScript, PostgreSQL	Turns a hold into a paid order (mock payment)	HPA 2–8
Redis 7.4	Deployment	Seat locks with TTL, sold-seat set	1
PostgreSQL 17	StatefulSet + PVC	Orders; primary key (event, seat)	1

3.1 How a seat is sold exactly once

1. **Hold.** Inventory runs `SET hold:<event>:<seat> <hold-id> NX EX 90` per seat. `NX` means only the first fan wins; the second gets HTTP 409. Multi-seat holds are all-or-nothing, and release uses a `WATCH/MULTI` compare-and-delete so one fan can never release another fan's lock.
2. **Pay.** Checkout re-reads the hold, charges the mock gateway, and inserts the order in a transaction. `PRIMARY KEY (event_id, seat_id)` on `order_seats` is the final safety net.
3. **Commit.** Checkout tells inventory to move the seats to the sold set. If the hold expired during payment the order is deleted and the fan is told they were not charged.
4. **Expire.** Abandoned holds disappear after the TTL and the seats return to sale automatically.

Because all state lives in Redis and PostgreSQL, the three services are stateless, which is what allows the HorizontalPodAutoscaler to add and remove pods freely during a surge.

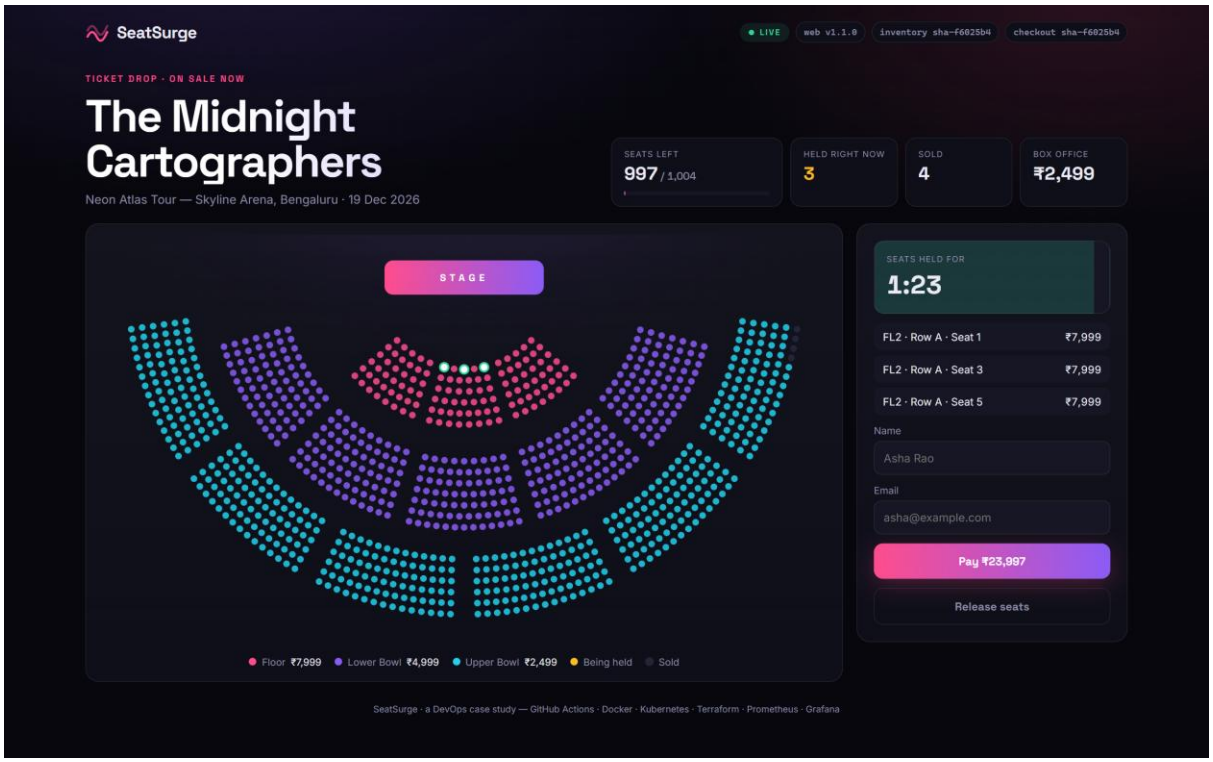


Figure 2. *Three floor seats held by a fan: 1:23 left on the 90-second hold. Header chips show the live build of each service.*

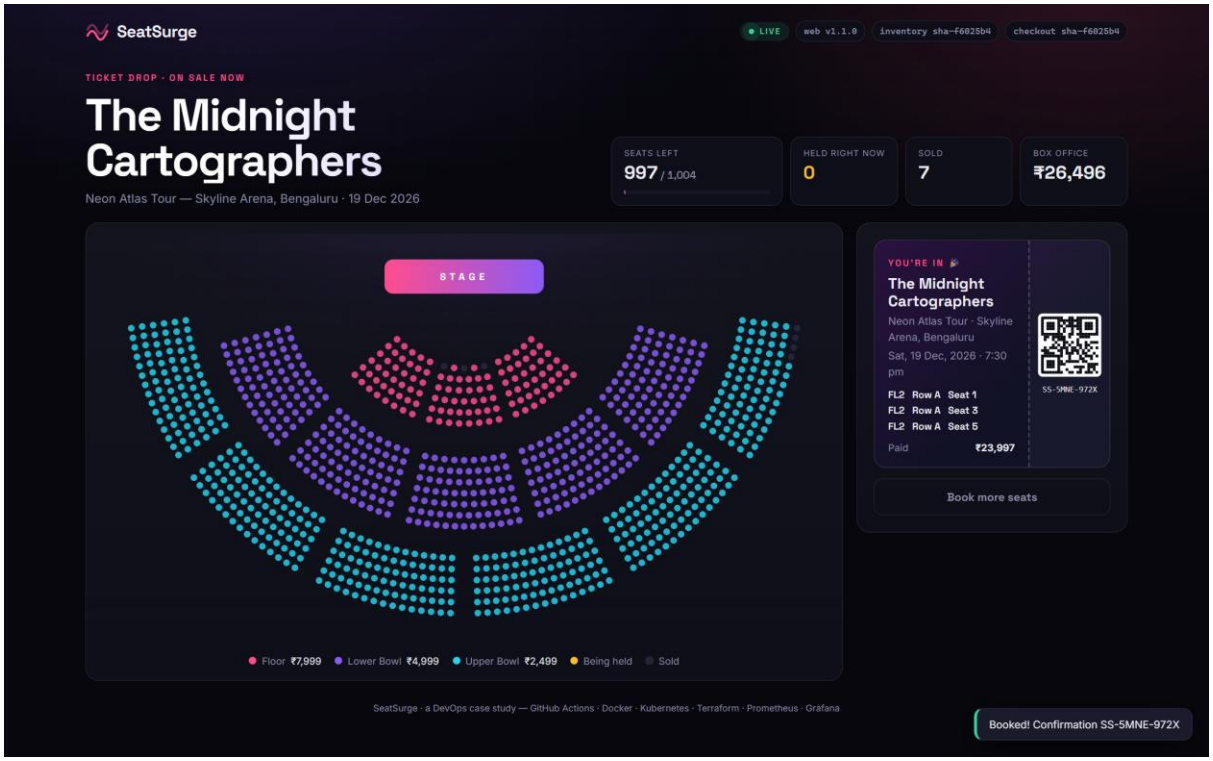


Figure 3. *After payment: the ticket stub with confirmation code. Sold seats turn dark on every fan's map within 1.5 s.*

4. Version control and collaboration

4.1 Branching strategy: GitFlow-lite

Two long-lived branches and short-lived topic branches. Everything reaches `develop` and `main` through pull requests; nothing is pushed directly.

Table 2. *Branch model*

Branch	Purpose	Merges into	Protection
main	Production. Every commit is deployed dev → staging → approval → prod.	—	PR required, 10 required checks, up-to-date, no force-push/delete, conversations resolved
develop	Integration branch, always deployable to staging. Default branch.	main via release PR	Same as main
feature/*, fix/*	One feature or bug fix	develop (squash)	—
hotfix/*	Urgent production fix, branched from main	main, then back-merged into develop	—

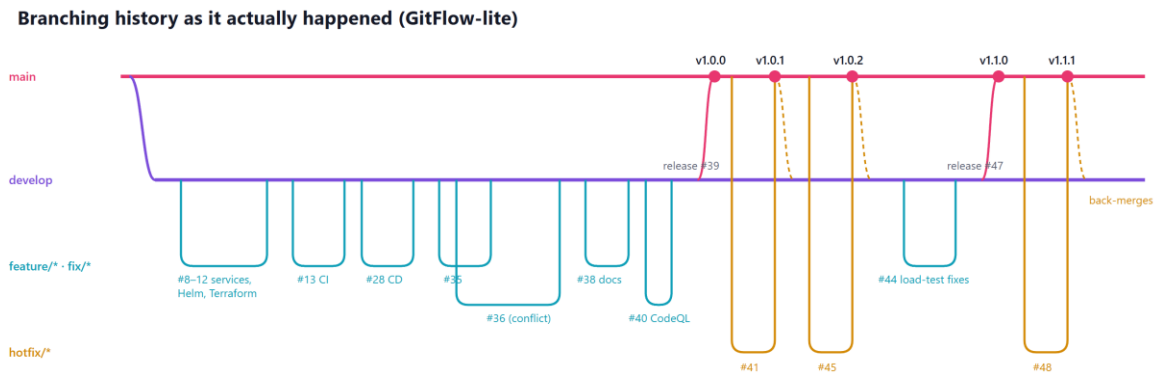


Figure 4. *The branching history as it actually happened: 8 feature/fix PRs, a deliberate conflict (#35/#36), two release PRs and three hotfixes, each back-merged.*

4.2 Pull-request workflow and team practices

- **Issues first.** Every feature and bug has an issue (11 in total, all closed by their PRs through `Closes #n`).
- **Conventional Commits** (`feat(inventory): …`, `fix(ci): …`) make history readable and drive generated release notes.
- **PR template** asks what/why, type of change, how it was tested and a checklist (no secrets, docs updated). **CODEOWNERS** requests the owner's review automatically.
- **Branch protection** requires every CI check to pass on a branch that is up to date with its target (strict mode), and all review conversations, including bot findings, to be resolved.
- **Auto-merge + squash** keeps `develop` linear; release PRs use merge commits so `main` records each release as one merge.

- **Dependabot** watches pip, npm, Docker, GitHub Actions and Terraform weekly; it opened 18 update PRs on its own, which flow through the same CI gates.

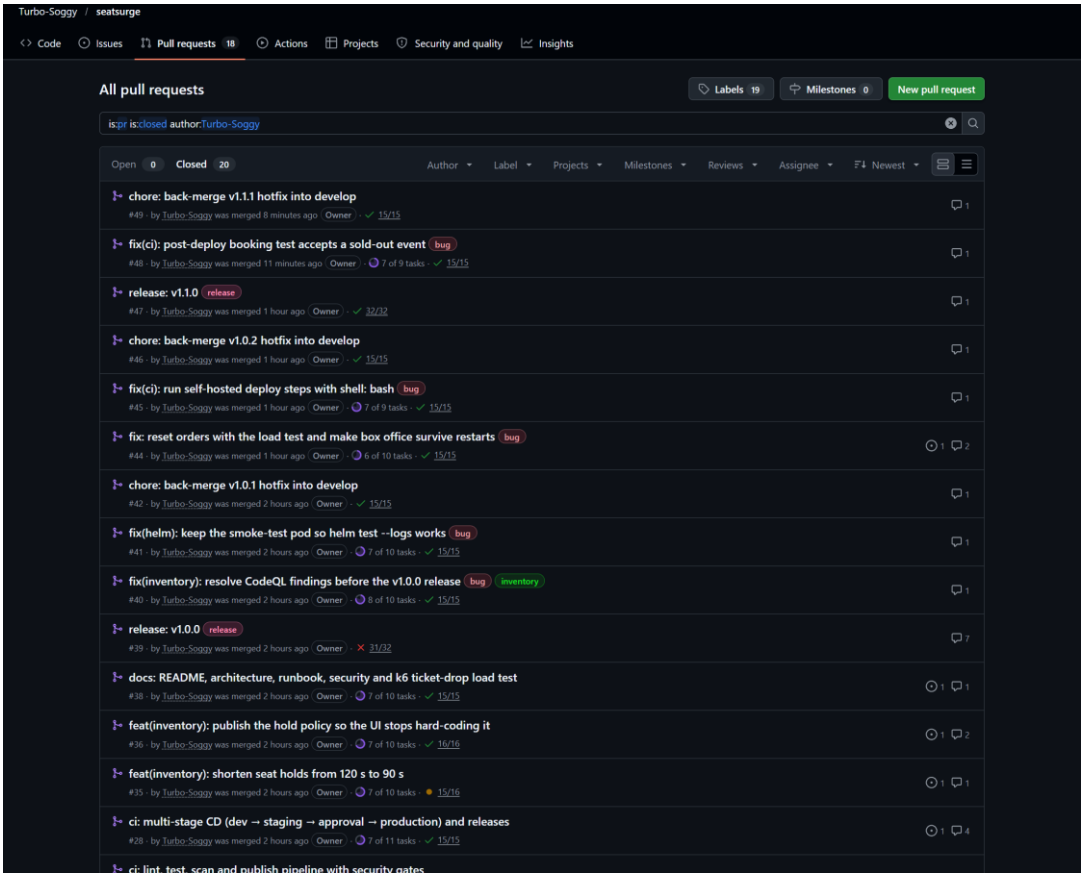


Figure 5. *20 merged pull requests: features, a fix PR, two releases, three hotfixes and their back-merges, each with green required checks.*

4.3 Merge-conflict case study

Two feature branches changed the seat-hold timeout in parallel. **#35** shortened it from 120 s to 90 s (in the service, Helm values, compose file and the UI text). **#36** made the API publish the hold policy so the UI stops hard-coding it. #35 merged first; GitHub then blocked #36 with “This branch has conflicts”.

```
$ git merge origin/develop
Auto-merging deploy/helm/seatsurge/values.yaml
CONFLICT (content): Merge conflict in deploy/helm/seatsurge/values.yaml
Auto-merging services/web/src/App.tsx
CONFLICT (content): Merge conflict in services/web/src/App.tsx
```

The branch was already shared and under review, so `develop` was **merged** into it rather than rebased. Each hunk was resolved by intent, not by picking a side:

Table 3. *Conflict resolution by intent*

File	develop (#35)	branch (#36)	Resolution
values.yaml	`holdTtlSeconds: 90`	policy comment + `120`	comment **and** `90`
App.tsx (copy)	hard-coded “90 seconds”	`{formatDuration(holdTtl)}`	API-driven copy (now renders “90 seconds” automatically)
App.tsx (fallback)	—	`DEFAULT_HOLD_SECONDS = 120`	**90**: no textual conflict, but semantically wrong after #35

The third row is the lesson: the most important fix was on a line Git never flagged. Tests (pytest 17, vitest 14) and `helm template` were re-run locally; CI then re-validated the merge result before auto-merge. The full write-up is in `docs/merge-conflict-case-study.md`.

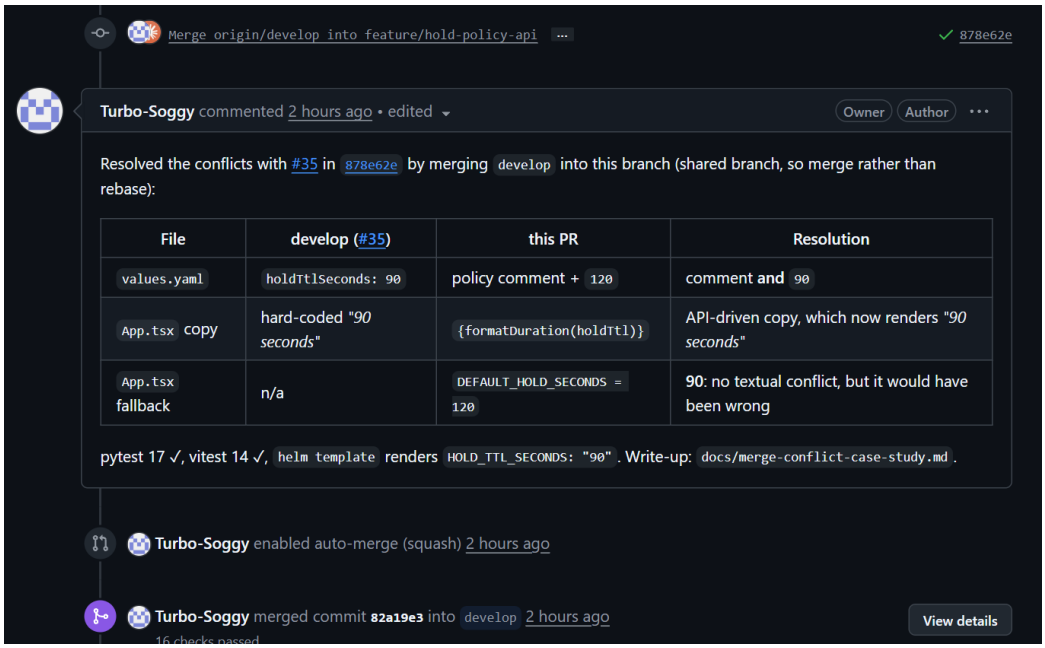


Figure 6. *The resolution documented on PR #36 for the reviewer, followed by auto-merge after 16 green checks.*

5. CI/CD pipeline automation

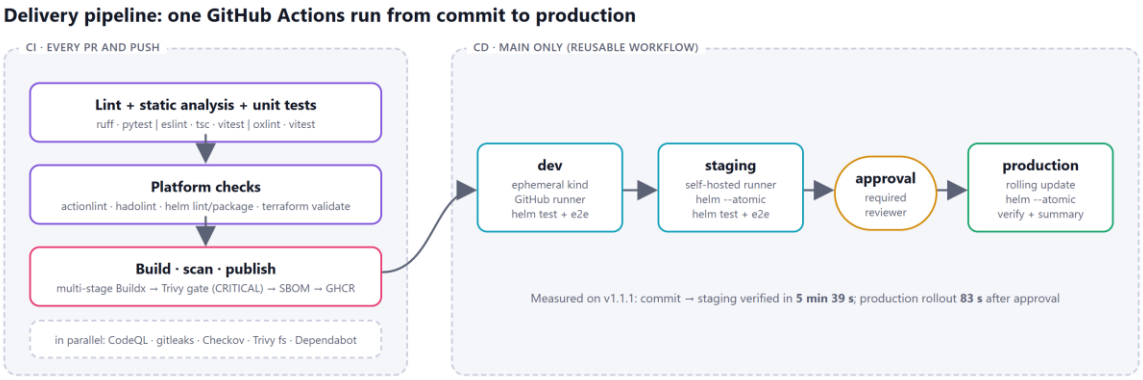


Figure 7. *The delivery pipeline. CD is a reusable workflow that CI calls on pushes to main, so one run shows the whole path from commit to production.*

5.1 Continuous integration (every PR and push)

Table 4. *CI stages*

Stage	What runs	Tools
Lint & static analysis	Python lint/format; ESLint and oxlint; TypeScript type-check; workflow lint; Dockerfile lint	ruff, eslint, oxlint, tsc, actionlint + shellcheck, hadolint
Unit tests + coverage	49 tests: inventory 17 (93 %), checkout 18 (95 % lines, 80 % gate), web 14. JUnit + coverage uploaded as artifacts	pytest + pytest-cov, vitest + Testing Library
Platform checks	`helm lint` + render for dev/staging/prod, `helm package`; `terraform fmt` + `validate` for both modules	Helm, Terraform
Build	Multi-stage Buildx builds with GitHub Actions layer cache; tests also run inside the build	Docker Buildx
Security gate	Trivy fails the build on any fixable CRITICAL CVE; CRITICAL/HIGH report to the Security tab as SARIF	Trivy (pinned by SHA)
Artifacts	SPDX SBOM per image; images pushed to GHCR as `sha-*`, branch, semver and `latest`; Helm chart `.tgz`; GitHub Release per tag	Syft, GHCR, release.yml
In parallel	SAST for Python, JS/TS and the workflows themselves; secret scan of full history; IaC scan; dependency scan	CodeQL, gitleaks, Checkov, Trivy fs

5.2 Continuous delivery (main only)

- **dev:** a throwaway kind cluster on a GitHub-hosted runner. The release images are pulled from GHCR, installed with Helm, verified with `helm test` and an end-to-end booking (hold → rival gets 409 → pay → seat sold), then destroyed.
- **staging:** the long-lived cluster, through a self-hosted runner on the cluster host (labels `self-hosted, seatsurge`). `helm upgrade --atomic` rolls back automatically if pods do not become ready.

- **production:** the GitHub environment has a **required reviewer**. The run pauses until a human approves, then performs the same atomic rolling update and verification and writes the Helm history to the run summary.
- **Rollback / redeploy:** `workflow_dispatch` with an `image_tag` input redeploys any published image; `helm rollback` restores the previous revision (Section 6.4).

Self-hosted runner safety on a public repository: the deploy jobs only run for pushes to `main` or a manual dispatch by a maintainer, never for pull requests; workflow runs from outside contributors need approval; and every third-party action is pinned to a commit SHA.

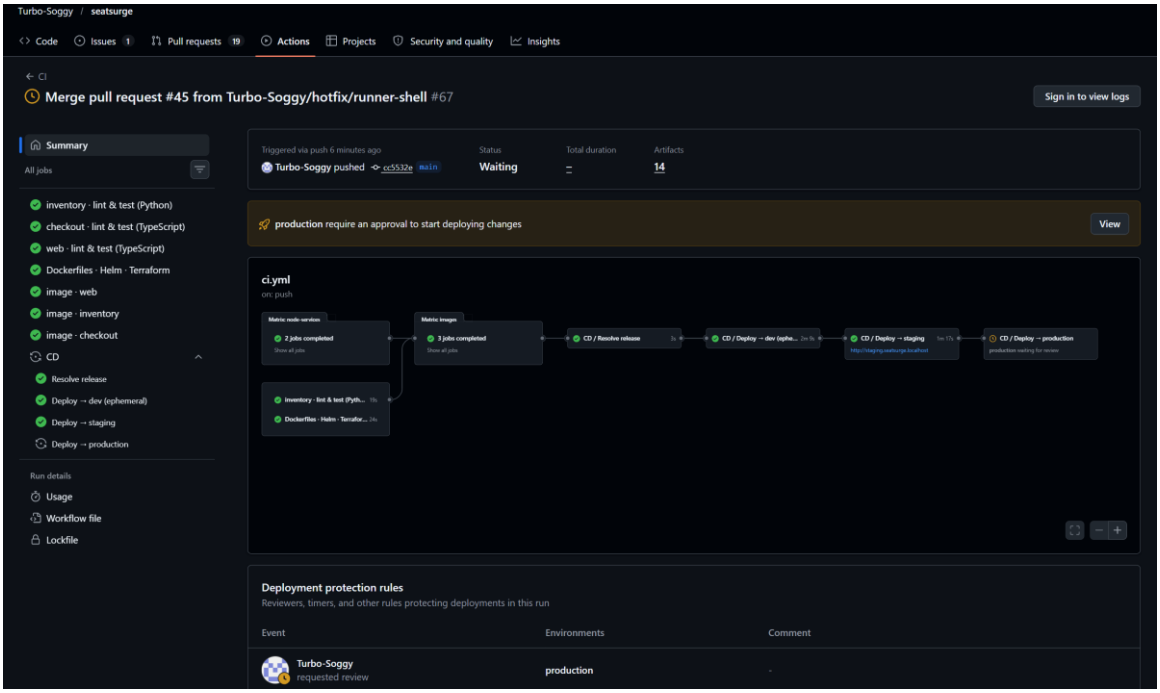


Figure 8. *Release paused at the production gate: CI, dev and staging are green, and production waits for the required reviewer.*

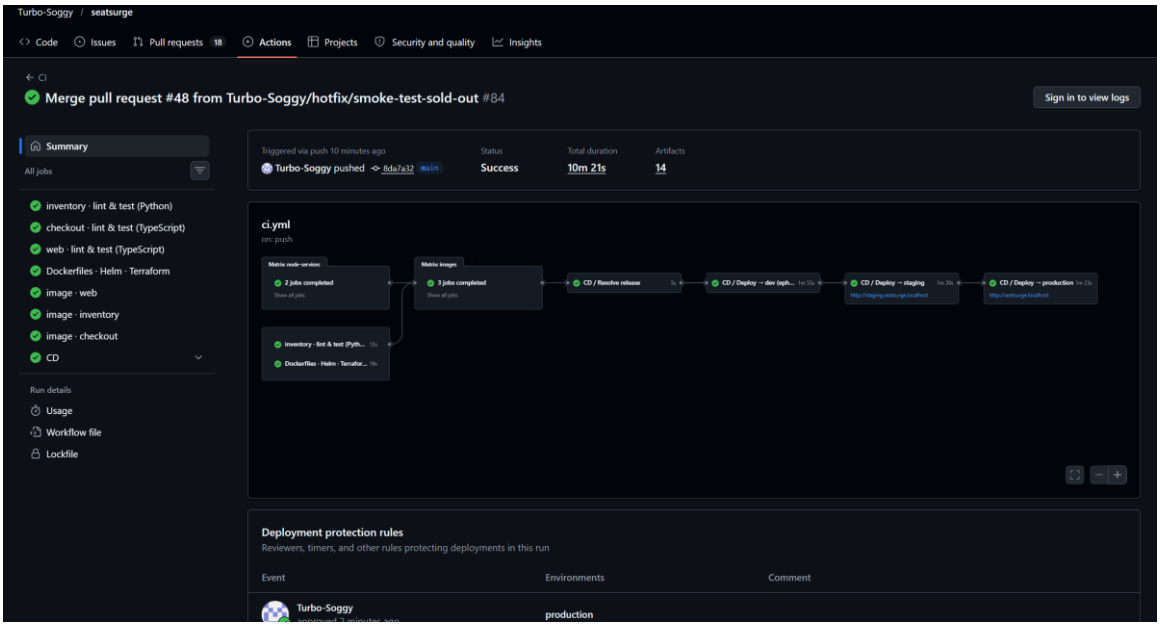


Figure 9. *The same pipeline after approval for v1.1.1: every stage green, 14 artifacts (test reports, coverage, SBOMs, Helm chart, web build).*

Table 5. *Measured pipeline timings*

Stage (v1.1.1 run)	Duration
Lint + unit tests (3 jobs, parallel)	13–19 s
Dockerfiles · Helm · Terraform	19 s
Build + Trivy + SBOM + push (3 images, parallel)	66–96 s
Deploy → dev (create cluster, install, test, destroy)	1 min 55 s
Deploy → staging (atomic upgrade, helm test, e2e)	1 min 30 s
Commit → staging verified	**5 min 39 s**
Deploy → production after approval	1 min 23 s

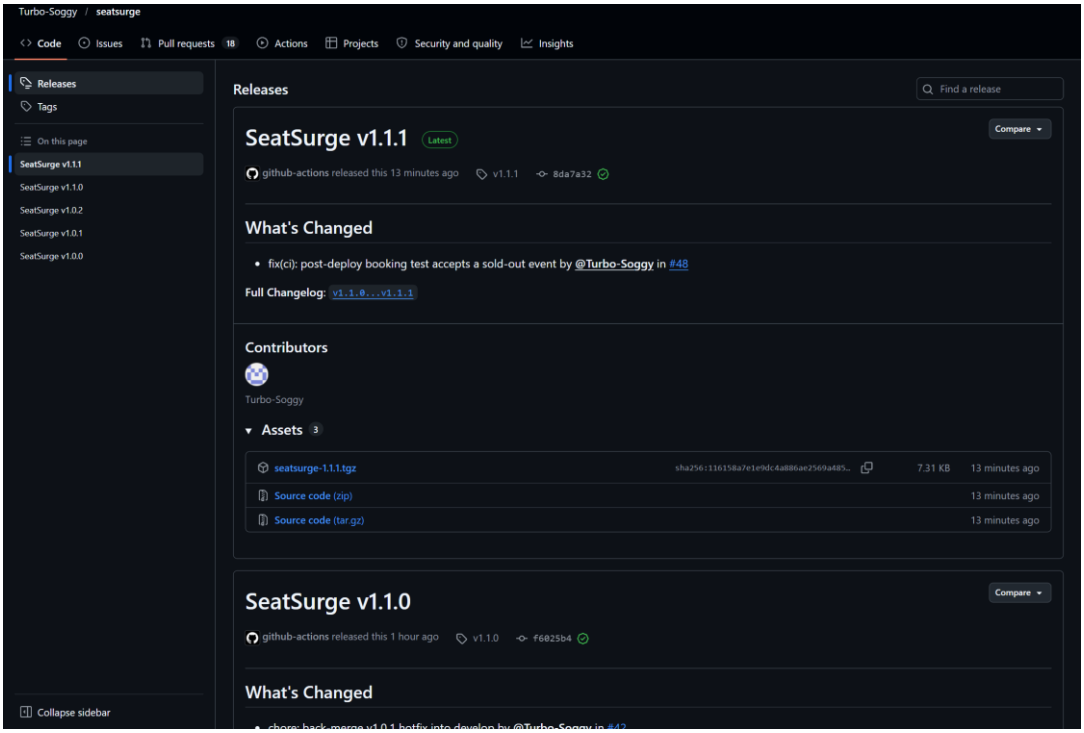


Figure 10. *Five releases published automatically by release.yml, each with the versioned Helm chart and generated notes.*

6. Containerization and orchestration

6.1 Optimized multi-stage Dockerfiles

Table 6. *Container images (*uncompressed size reported by Docker)*

Image	Stages	Runtime base	User	Size*
inventory	builder (wheels) → test (ruff + pytest) → runtime	python:3.13-slim	uid 10001	253 MB
checkout	deps → build (lint, typecheck, test, tsc, prune) → runtime	distroless nodejs24-debian13	uid 65532	240 MB
web	deps → build (vite) → runtime	nginx-unprivileged 1.29-alpine	uid 101	116 MB

- Tests run **inside** the build, so an image cannot exist for code that fails its tests.
- Runtime images contain no compilers, package managers or shells (distroless), and run as **numeric non-root UIDs** so Kubernetes `runAsNonRoot` can verify them.
- Pinned base images, `.dockerignore`, ordered layers for cache hits, JSON-form `HEALTHCHECK`, and `apk upgrade` / Debian 13 bases chosen after Trivy found CRITICAL CVEs in older bases.
- Images are published to **GitHub Container Registry**; the Azure module provisions **ACR** with `AcrPull` for the cluster identity (no registry passwords).

6.2 Kubernetes objects (Helm chart)

Table 7. *Kubernetes resources in the chart*

Object	Configuration
Deployment ×3	`RollingUpdate maxSurge 1 / maxUnavailable 0`; readiness + liveness probes; preStop drain; resource requests/limits; topology spread; read-only root FS; dropped capabilities; seccomp RuntimeDefault
Service ×5	ClusterIP for web, inventory, checkout, Redis, Postgres
Ingress	Traefik; `/` → web, `/api/inventory`, `/api/checkout`; host per environment
HorizontalPodAutoscaler ×3	autoscaling/v2 on CPU: inventory 2–10, checkout 2–8, web 2–4
PodDisruptionBudget ×3	minAvailable 1 so node drains never take a service down
StatefulSet + PVC	PostgreSQL with persistent storage
ConfigMap / Secret / SealedSecret	Non-secret config; generated password for dev/staging; encrypted SealedSecret for prod
ServiceMonitor ×2, PrometheusRule	Scrape config and 7 alert rules shipped with the app
helm test Pod	In-cluster smoke test: web, readiness of both APIs, proxy routing, metrics

6.3 Scaling and zero-downtime rolling update under load

The decisive test: release v1.1.0 was approved for production while k6 drove 150 virtual fans at the site. A probe also hit both APIs every 0.5 s to record HTTP status and the version answering.

Table 8. *Rolling update of v1.1.0 under a 150-fan load test*

Measurement	Result
HTTP requests during the rollout window	**14,330**
Failed requests	**0 (0.00 %)**
k6 checks passed	14,144 / 14,144
Latency avg / p90 / p95	14.4 ms / 14.6 ms / 23.5 ms
Version probes (every 0.5 s)	476, all HTTP 200; versions switched sha-cc5532e → sha-f6025b4 mid-load
Orders confirmed / hold conflicts (409)	530 / 679
Outcome	Sold out: 1,004 seats, ₹41,18,996, no seat sold twice
Autoscaling	inventory 2 → 10 pods within ~60 s, back to 2 after the surge

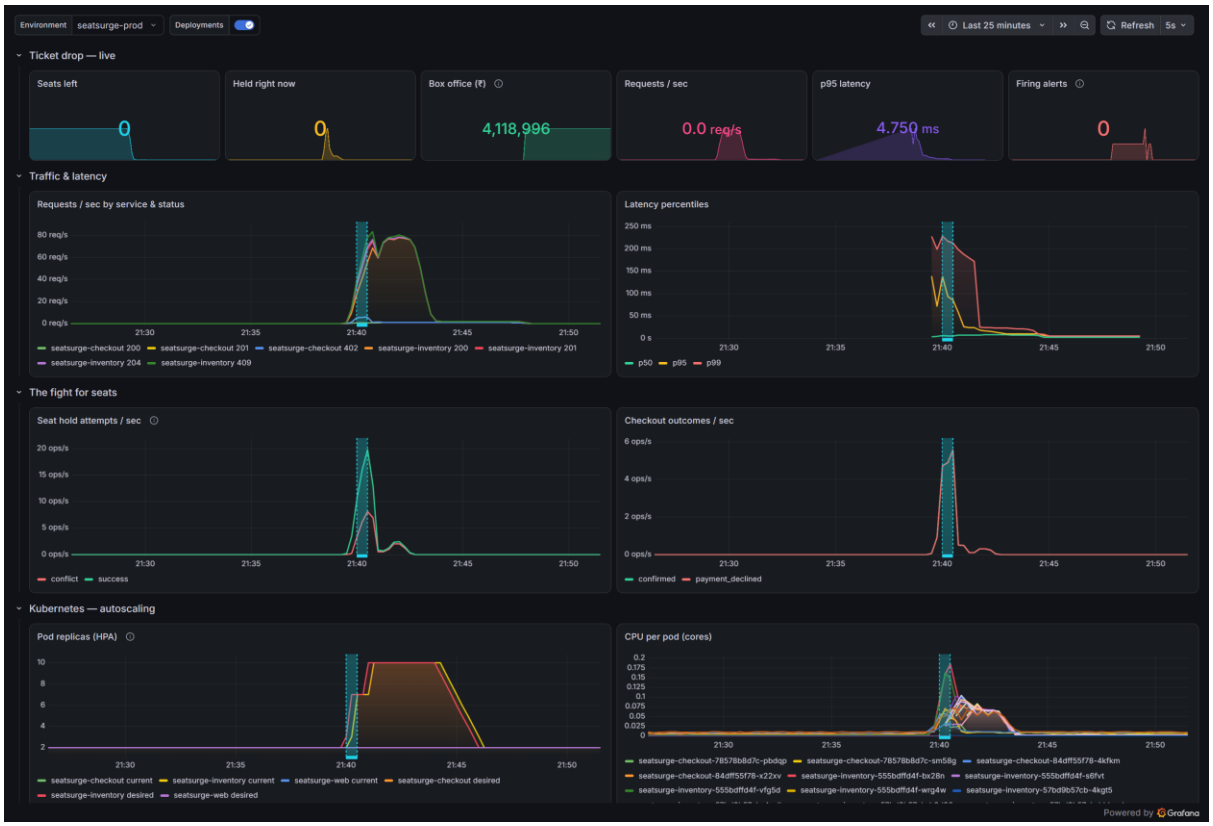


Figure 11. *Grafana during the v1.1.0 release: the shaded band is the rollout (a new ReplicaSet), traffic and checkouts peak inside it, latency stays low, and the HPA scales inventory to 10 pods.*

6.4 Rollback

`helm rollback` was exercised on staging: back to the previous release, verified, and forward again.

```
$ helm rollback seatsurge 3 -n seatsurge-staging --wait
Rollback was a success! Happy Helming!
live: inventory=sha-cc5532e checkout=sha-cc5532e
$ helm test seatsurge -n seatsurge-staging        → Phase: Succeeded
$ helm rollback seatsurge 4 -n seatsurge-staging --wait   # roll forward
live: inventory=sha-f6025b4 checkout=sha-f6025b4
REVISION  CHART            DESCRIPTION
5         seatsurge-1.0.0  Rollback to 3
6         seatsurge-1.1.0  Rollback to 4
```

Each rollback completed in about 15 seconds. In the pipeline, `helm upgrade --atomic` performs the same rollback automatically when a release does not become ready.

7. Infrastructure as code, cloud, monitoring and security

7.1 Terraform: the whole platform from one command

`infra/terraform/local` uses the kind, Helm and Kubernetes providers to build the complete platform with one `terraform apply`. Every chart version is pinned in `variables.tf`.

Table 9. *Local platform provisioned by Terraform*

Resource	Purpose
kind cluster	1 control-plane + 2 workers, ports 80/443 mapped to the host
Namespaces	monitoring, seatsurge-staging, seatsurge-prod
Traefik 42.3	Ingress controller (ingress-nginx was retired upstream in 2026)
metrics-server	CPU metrics for the HPAs
Sealed Secrets	Controller that decrypts SealedSecrets in-cluster
kube-prometheus-stack 93.1	Prometheus, Alertmanager, Grafana (with ingress), node-exporter, kube-state-metrics
Loki 7.5 + Grafana Alloy 1.15	Log storage and log shipping from every pod
Dashboard ConfigMap	The SeatSurge dashboard, generated from code (`monitoring/dashboards/generate.py`)

Cloud target (Azure). `infra/terraform/azure` provisions a VNet, an **AKS** cluster (Azure CNI Overlay + Cilium, Entra ID RBAC, workload identity, three zones, an autoscaling 1–5 node app pool), **ACR** with `AcrPull` for the kubelet identity, **Key Vault** with RBAC plus a federated identity so checkout reads its database password without a stored secret, and **Log Analytics** with Container Insights. It is validated by `terraform validate` in every CI run; `terraform plan/apply` needs an Azure subscription (`az login`).

7.2 Monitoring, logging and alerting

- **Metrics.** Both APIs expose Prometheus metrics: RED metrics per route (`seatsurge_http_requests_total`, latency histogram) and business metrics (hold attempts by result, seats available/held, orders by outcome, and box office read from PostgreSQL).
- **Dashboard as code.** 18 panels: headline numbers, traffic and latency percentiles, the fight for seats, checkout outcomes, HPA replicas, CPU per pod, and live application logs from Loki, with rollout annotations.
- **Logs.** Structured JSON logs (pino, Python logging) shipped by Alloy to Loki and queryable next to the metrics.
- **Alerts.** Seven PrometheusRule alerts, each linking to a runbook section: HighErrorRate, HighLatencyP95, TargetDown, TicketFrenzy, SoldOut, HPAMaxedOut, PodCrashLooping. SoldOut fired during the load tests and HPAMaxedOut went pending at the 10-pod ceiling.

Observability also found two bugs (Section 9): the original box-office panel summed an in-process counter that dropped whenever the autoscaler removed a pod, and the rollout annotation fired on every scale event. Both were fixed in PR #44:



Figure 12. *Before the fix: box office shows ₹4,35,851 after scale-down, and rollout markers appear for HPA scaling as well as the real rollout.*



Figure 13. *After the fix: box office read from PostgreSQL shows the true ₹41,18,996 and a single marker for the real rollout.*

7.3 Secrets management

- **Production database password:** a **SealedSecret** encrypted with the cluster's public key and committed safely; only the in-cluster controller can decrypt it (`scripts/seal-secret.ps1`).
- **Dev/staging:** a random password generated by Helm on first install and preserved across upgrades with `lookup`.
- **Pipeline credentials:** only the short-lived `GITHUB_TOKEN` with least-privilege permissions per job; no long-lived secrets are stored in the repository or its settings.
- **Azure:** Key Vault + workload identity federation, so pods authenticate without any stored secret.
- **Guard rails:** gitleaks scans the full git history on every PR; reviewed false positives are allowlisted narrowly by pattern.

7.4 Security scanning (DevSecOps)

Table 10. *Security controls*

Layer	Tool	Policy	Outcome in this project
Container images	Trivy	Fail build on fixable CRITICAL; report CRITICAL/HIGH	0 CRITICAL in shipped images; base images upgraded twice to clear CRITICAL CVEs
Source code (SAST)	CodeQL (Python, JS/TS, Actions)	Findings become PR review threads; merge blocked until resolved	Fixed log injection, side-effect in assert, unclosed file, 14 unpinned actions, untrusted checkout
Secrets in history	gitleaks	Required check	Clean; one reviewed false positive allowlisted

Layer	Tool	Policy	Outcome in this project
Infrastructure as code	Checkov	Soft-fail, results in Security tab	Findings triaged; NetworkPolicies listed as future work
Dependencies	Trivy fs, Dependabot	Weekly updates as PRs	18 Dependabot PRs opened automatically
Supply chain	SHA-pinned actions, SBOM (Syft)	Reproducible builds, inventory of every package	SPDX SBOM attached to every image build

8. Results

Table 11. *Summary of results*

Rubric area	Evidence produced
Version control & collaboration	20 PRs merged through protected branches, 11 issues, a documented merge conflict, 3 hotfixes with back-merges, 18 Dependabot PRs
CI/CD automation	5 releases (v1.0.0 → v1.1.1) delivered by the pipeline; commit → staging in 5 min 39 s; manual approval gate; auto-rollback
Containers & orchestration	3 multi-stage non-root images; HPA 2 → 10 under load; rolling update with 0 failed requests in 14,330; helm rollback in ~15 s
IaC, cloud, monitoring, security	Platform from one terraform apply; AKS module validated; 18-panel dashboard, Loki logs, 7 alerts; SealedSecret; 5 scanners
Documentation	README with diagrams, architecture/runbook/security/branching/case-study docs, this report and the slide deck

Table 12. *k6 load tests*

Load test	Fans	Requests	Failed	p95	Outcome
Run 1 (v1.0.0)	150	13,145	0.00 %	15.7 ms	sold out, HPA 2→10
Run 2 (stale seat view)	200	24,943	0.00 %	—	3,950 conflicts; exposed reset bug
Run 3 (v1.1.0 rollout)	150	14,330	0.00 %	23.5 ms	sold out during a rolling update

9. Defects caught by the pipeline, and how they were fixed

A pipeline earns its keep by catching problems before users do. Every row below happened during this project and was fixed through a pull request.

Table 13. *Defects found and fixed*

#	Caught by	Problem	Fix (PR)
1	GitHub workflow parser	Unquoted `:` in a YAML scalar; CI never started	Quote the step; add actionlint to CI (#13)
2	CI	`trivy-action@0.33.1` tag no longer existed after upstream retagging	Pin by commit SHA (#13)
3	gitleaks	False positive on a Kubernetes secret key name	Narrow allowlist with a comment (#13)
4	CI	pytest could not import the app outside the venv	`pythonpath` in pyproject (#13)
5	CodeQL (Actions)	14 third-party actions on mutable tags	Pin all to SHAs; Dependabot keeps them current (#13)
6	CodeQL (Actions)	Untrusted checkout in a workflow_run CD	CD as reusable workflow called from CI (#28)
7	CodeQL (Python)	Log injection; side-effect in assert	Log server-generated id; restructure tests (#40)
8	CD dev stage	`helm test --logs` failed: pod deleted on success	Hotfix v1.0.1 (#41)
9	CD staging stage	Runner could not parse quoted custom shell path	Hotfix v1.0.2: `shell: bash` (#45)
10	Load test + smoke test	Reset cleared Redis but not Postgres; checkout 409s	Guarded checkout reset; k6 resets both (#44)
11	Grafana	Box office fell on scale-down; markers on HPA scaling	DB-backed gauge; ReplicaSet-based markers (#44)
12	CodeQL (Python)	Dashboard generator left a file open	`with` block (#44)
13	CD production stage	Post-deploy booking test failed when the event sold out	Hotfix v1.1.1: verify 409 + consistency instead (#48)

10. Rubric alignment

Table 14. *Mapping to the evaluation rubric*

Criterion (7 marks each)	Where it is demonstrated
DevOps ecosystem, version control & collaboration	Section 4; `docs/branching-strategy.md`; `docs/merge-conflict-case-study.md`; PR/issue templates, CODEOWNERS, Dependabot, branch protection
CI/CD pipeline automation	Section 5; `.github/workflows/ci.yml`, `cd.yml`, `release.yml`, `security.yml`, `codeql.yml`
Containerization & orchestration	Section 6; `services/*/Dockerfile`; `deploy/helm/seatsurge`
IaC, cloud deployment & monitoring	Section 7; `infra/terraform/local`, `infra/terraform/azure`; `monitoring/`; `docs/security.md`, `docs/runbook.md`
Documentation & presentation	README, `docs/`, this report, `SeatSurge-Slides.pptx`

Pipeline execution: live demonstration script

1. Open a small PR (e.g. change a label in the UI) and show CI running: lint, tests, image build, Trivy, CodeQL.
2. Merge a release PR into `main`; show the run graph reaching dev and staging, then the production approval gate.
3. Start `./scripts/demo.ps1` (k6, 150 fans) and approve production; watch Grafana: traffic, HPA scaling, the rollout band, and the version chips changing in the app with no errors.
4. Run `helm rollback` on staging and `helm history` to show instant rollback.

11. Conclusion and future work

SeatSurge shows the full DevOps loop working on a problem where it matters. Correctness comes from atomic locks and a database constraint; elasticity from stateless services and autoscaling; safe change from a gated, automated pipeline with atomic rollouts and instant rollback; and visibility from metrics, logs, dashboards and alerts designed around the business. The most valuable outcome was not that everything worked first time, but that **the pipeline caught 13 real defects**, from YAML syntax to a semantic merge conflict to a data-consistency bug that only appeared under load, and each fix went through the same process as a feature.

Future work

- Apply the Azure module and run the same CD pipeline against AKS with OIDC federation from GitHub Actions (no stored cloud credentials).
- Add NetworkPolicies (default-deny plus explicit allow) to close the main Checkov finding.
- GitOps with Argo CD for pull-based delivery, and progressive delivery (canary with automated analysis on the Prometheus SLOs).
- A waiting-room queue in front of inventory, so the platform sheds load gracefully beyond the HPA ceiling.

References

- Repository: github.com/Turbo-Soggy/seatsurge (code, workflows, docs, run history).
- Kubernetes documentation: Deployments, HorizontalPodAutoscaler, PodDisruptionBudget.
- Helm documentation: chart tests, `--atomic`, rollback.
- GitHub Actions documentation: environments and required reviewers, reusable workflows, security hardening.
- Prometheus Operator, Grafana Loki and Grafana Alloy documentation.